

Principal Component Analysis (PCA) Project

Ali Sinan Dastan, Antonio Perazza, Krystof Kouril

1 Brief Introduction to the Project

Principal component analysis (PCA) is one of the oldest and widely known techniques of multivariate analysis [1]. Like many multivariate methods, it was not widely used until the advent of electronic computers, but it is now well entrenched in virtually every statistical computer package.

The central idea of principal component analysis is to reduce the dimensionality of a data set in which there are a large number of interrelated variables, while retaining as much as possible of the variation present in the data set [2]. This reduction is achieved by transforming to a new set of variables, the principal components, which are uncorrelated, and which are ordered so that the first few retain most of the variation present in all of the original variables.

In this research, the data from the World Bank Group’s databank of “World Development Indicators” [3] for the year 2017 were utilized to conduct a PCA, in order to reflect and gain insights upon the relationships and correlations between certain factors such as industrialization, unemployment with a country’s level of development. These factors were initially plotted on a 2 dimensional data plane to configure the principal component 1 (PC1) and principal component 2 (PC2, perpendicular to PC1). Ultimately, the aim of our project became to establish whether these PCs have a correlation or relationship with the development of countries.

2 Mathematical Foundation of PCA

As seen, the aim of PCA is reducing the dimensionality of a dataset with multiple interrelated variables while retaining as much variability as possible.

Formally, consider a random vector $\mathbf{x} = (x_1, x_2, \dots, x_p)^T$ with mean vector $\boldsymbol{\mu}$ and covariance matrix Σ . PCA seeks linear combinations of the original variables, known as principal components (PCs), defined as $z_k = \boldsymbol{\alpha}_k^T \mathbf{x}$, such that the variance $\text{Var}(z_k)$ is maximized under the constraints $\|\boldsymbol{\alpha}_k\| = 1$ and $\boldsymbol{\alpha}_k^T \boldsymbol{\alpha}_j = 0$ for all $j < k$. This optimization leads to an eigenvalue problem: $\Sigma \boldsymbol{\alpha}_k = \lambda_k \boldsymbol{\alpha}_k$, where λ_k and $\boldsymbol{\alpha}_k$ are the k -th largest eigenvalue and its corresponding eigenvector of Σ . The eigenvalues represent the variances of the PCs, i.e. $\text{Var}(z_k) = \lambda_k$, and the total variance in the data is $\sum_{k=1}^p \lambda_k$. The transformation matrix $A = [\boldsymbol{\alpha}_1, \dots, \boldsymbol{\alpha}_p]$ orthogonally diagonalizes Σ , such that $A^T \Sigma A = \Lambda$, a diagonal matrix of eigenvalues. Consequently, PCs are uncorrelated and ordered by decreasing variance. Geometrically, this transformation aligns the coordinate axes with the directions of maximal data spread. In practice, the unknown covariance matrix Σ is estimated by the sample covariance matrix S , and the same eigen-decomposition process is applied. This framework ensures that the first few PCs capture the bulk of the data variability, enabling an effective lower-dimensional representation. Importantly, if data are standardized, the correlation matrix may be used instead of Σ , particularly when variables differ in scale. PCA thus provides both an algebraic and geometric insight into data structure, foundational for numerous applications in statistics and machine learning.

3 Practical Application

As mentioned in the first section of this report we have decided to apply PCA on a dataset of World Development indicators. In the end we managed to achieve a dual implementation. The first one was simply done by using the built-in *prcomp()* R function (we used this version for the interpretation that will follow, see Appendix B). The second one, instead, was more hands-on, conducting the PCA step by step through individual operations, this implementation is recorded in Appendix C with its mathematical details and intermediate results detailed in Appendix A.

Through PCA of 12 different indicators we ended up with two main PCs accounting for 51.9 % and 10.9 % of the variance respectively, the first two PCs are therefore responsible for about 63 % of the total variance which is slightly less than the ideal >70 % suggested by I. T. Jolliffe [2]. Through our analysis we have concluded that PC1 indicates the general economic development of the country, where the higher the PC1 classification, the more economically developed the country is. This is the case, as we can see in Figure 1, since it is positively loaded by variables such as GDP per capita, internet usage, or life expectancy. Furthermore, it is negatively loaded by inflation and the role of agriculture in the economy. Mathematically, this represents the direction of maximum variance in the 12-dimensional space. The PC1 classification is headed by countries like Qatar, Singapore, and Kuwait, countries with barely any agriculture and high GDP per capita due to their stable economic conditions. On the other hand, the PC1 ranking is tailed by third world countries such as Chad or Central African Republic. Figure 2 projects the countries onto the two PCs and colour codes them based on continent. From this distinction we can notice patterns such as generally high ranking of European countries and the contrary for African. An interesting observation is also that Asia has a few very strong performers but overall is quite dispersed along the range.

In terms of PC2, this is a supplemental economic structure with positive loadings for unemployment and negative loading for CO2 emissions and industry importance. In Figure 2 the values were flipped to aid intuition. As PC2 accounts for merely 10.9 % of the variance in the 12-dimensional space its significance is lesser than PC1. Nevertheless, the country ranking based on this measure can offer valuable additional insights. Notable examples of this measure are Marshall Islands, and West Bank and Gaza on the lower end known for their lack of industry and high unemployment, and on the higher end oil countries such as Qatar or Kuwait with minimal unemployment and strong oil industry with high CO2 emissions. All these can be observed in Figure 2.

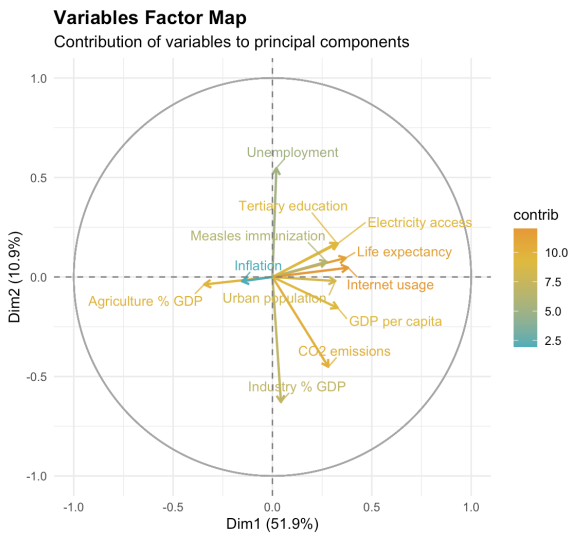


Figure 1: Variables Factor Map

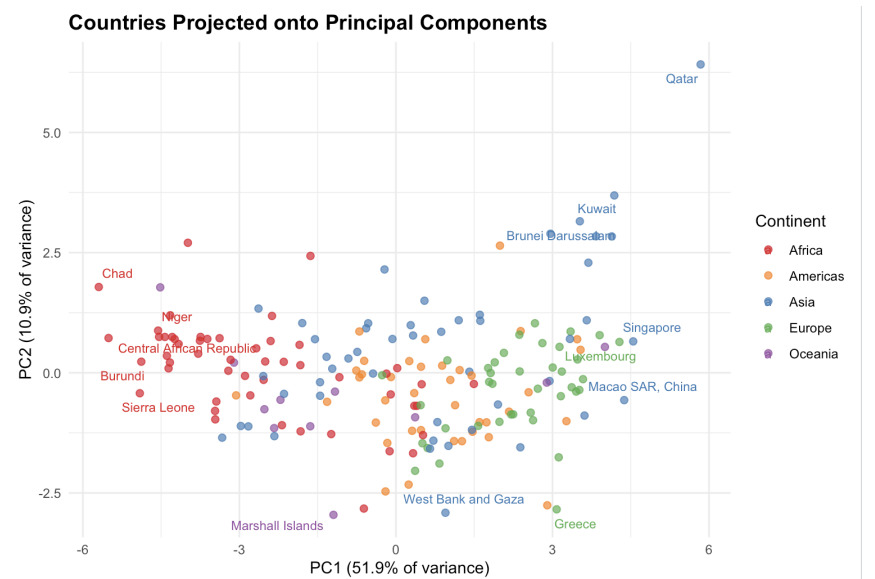


Figure 2: Country projection on first two PCs

References

- [1] Hervé Abdi. “Principal Component Analysis”. In: *WIREs Computational Statistics* 2.4 (2010), pp. 433–459.
- [2] Ian T. Jolliffe. *Principal Component Analysis*. Springer, 2002.
- [3] World Bank. *World Development Indicators*. Accessed: 2025-04-13. 2024. URL: <https://databank.worldbank.org/source/world-development-indicators>.

Appendices

A Mathematical Details of PCA Implementation

A.1 Standardization Values

Standardization transforms each variable to have zero mean and unit variance. For variable j , each value x_{ij} is transformed as $z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$, where μ_j is the mean and σ_j is the standard deviation. This ensures all variables contribute equally to the analysis regardless of their original scale.

A.1.1 Means of Key Indicators

Indicator	Mean
GDP per capita (PPP)	23,044.27
Life expectancy	72.04
Tertiary education	40.20
Measles immunization	86.97
Urban population (%)	57.96
CO2 emissions (per capita)	4.37

Table 1: Means of selected indicators before standardization

A.1.2 Standard Deviations of Key Indicators

Indicator	Standard Deviation
GDP per capita (PPP)	23,854.39
Life expectancy	7.40
Tertiary education	28.17
Measles immunization	12.54
Urban population (%)	21.89
CO2 emissions (per capita)	5.36

Table 2: Standard deviations of selected indicators before standardization

A.2 Correlation Matrix

For standardized data, the covariance matrix equals the correlation matrix. Element c_{jk} represents the Pearson correlation coefficient between variables j and k : $c_{jk} = \frac{1}{n-1} \sum_{i=1}^n z_{ij} z_{ik}$. This matrix captures the linear relationships between all pairs of variables.

A.2.1 First 3×3 Section of Correlation Matrix

	GDP per capita	Life expectancy	Tertiary education
GDP per capita	1.0000000	0.7281789	0.6020432
Life expectancy	0.7281789	1.0000000	0.7588756
Tertiary education	0.6020432	0.7588756	1.0000000

Table 3: Partial view of the correlation matrix (3×3 section)

A.3 Eigenvalues

Eigenvalues λ_j of the correlation matrix represent the amount of variance captured by each principal component. They are derived from the characteristic equation $\det(C - \lambda I) = 0$. Larger eigenvalues indicate components that capture more of the dataset's variability.

Eigenvalue	Value
λ_1	6.2227542
λ_2	1.3034818
λ_3	1.1143101
λ_4	0.8492074
λ_5	0.7914222
λ_6	0.4461775

Table 4: First six eigenvalues of the correlation matrix

A.4 Variance Explained

Each eigenvalue's proportion of variance is calculated as $\frac{\lambda_j}{\sum_{i=1}^p \lambda_i} \times 100\%$. The cumulative proportion shows how much total variance is explained by the first k components: $\frac{\sum_{j=1}^k \lambda_j}{\sum_{i=1}^p \lambda_i} \times 100\%$. This helps determine how many components to retain.

PC	Eigenvalue	Proportion (%)	Cumulative (%)
1	6.2227542	51.856285	51.85629
2	1.3034818	10.862348	62.71863
3	1.1143101	9.285918	72.00455
4	0.8492074	7.076728	79.08128
5	0.7914222	6.595185	85.67646
6	0.4461775	3.718145	89.39461

Table 5: Proportion and cumulative variance explained by each principal component

A.5 Principal Component Loadings

Loadings are the elements of eigenvectors $\mathbf{v}_j$ corresponding to eigenvalues λ_j . Each loading v_{ij} represents the correlation between variable i and principal component j for standardized data. They define the transformation from original variables to principal components.

A.5.1 PC1 Loadings (First 5 Variables)

Variable	PC1 Loading
GDP per capita	0.3285558
Life expectancy	0.3689285
Tertiary education	0.3238428
Measles immunity	0.2727875
Urban population	0.3168977

Table 6: Loadings of the first principal component (first 5 variables)

A.5.2 PC2 Loadings (First 5 Variables)

Variable	PC2 Loading
GDP per capita	-0.15564488
Life expectancy	0.09587534
Tertiary education	0.17033009
Measles immunity	0.07662605
Urban population	-0.02023503

Table 7: Loadings of the second principal component (first 5 variables)

A.6 Top Contributors

The magnitude of a loading $|v_{ij}|$ indicates the importance of variable i to component j . The sign indicates the direction of correlation. Variables with large absolute loadings are the primary “contributors” to that component, defining what the component represents conceptually.

A.6.1 Top Contributors to PC1

Indicator	Loading	Direction	Interpretation
Internet users	0.3808	Positive	Higher internet usage → More developed
Life expectancy	0.3689	Positive	Higher life expectancy → More developed
Agriculture value added	-0.3437	Negative	Lower agriculture % → More developed
Access to electricity	0.3290	Positive	Higher electricity access → More developed
GDP per capita	0.3286	Positive	Higher GDP → More developed

Table 8: Top five contributors to the first principal component

A.6.2 Top Contributors to PC2

Indicator	Loading	Direction
Industry value added	-0.6291	Negative
Unemployment	0.5472	Positive
CO2 emissions per capita	-0.4514	Negative
Tertiary education	0.1703	Positive
Access to electricity	0.1682	Positive

Table 9: Top five contributors to the second principal component

A.7 Principal Component Scores

Scores are calculated as $\mathbf{Z} = \bar{\mathbf{X}}\mathbf{V}$, where $\bar{\mathbf{X}}$ is the standardized data matrix and $\mathbf{V}$ is the matrix of eigenvectors. Each score z_{ij} represents the projection of observation i onto principal component j . Geometrically, scores are the coordinates of observations in the new PC space.

A.7.1 First 6 Countries on Principal Components

Country	PC1	PC2	PC3
Afghanistan	-3.3299696	1.3498049	0.46405583
Albania	0.6070019	1.5652507	0.01292625
Algeria	0.4931988	0.2384581	-1.15875621
Angola	-3.7445626	-0.7464829	-4.59912809
Antigua and Barbuda	1.2657188	1.4233817	0.28274530
Argentina	2.1672303	0.8087823	0.15808340

Table 10: PC scores for the first six countries in the dataset

A.8 Countries at Extremes

Extreme values on a principal component identify observations that strongly embody what that component represents. With the corrected signs, countries with high positive PC1 scores are the most developed while those with high negative scores are the least developed.

A.8.1 Highest PC1 Values (Most Developed)

Country	PC1 Score
Qatar	5.8394
Singapore	4.5473
Macao SAR, China	4.3745
Luxembourg	4.2837
Kuwait	4.1849

Table 11: Countries with highest PC1 scores (most developed)

A.8.2 Lowest PC1 Values (Least Developed)

Country	PC1 Score
Chad	-5.6942
Central African Rep.	-5.5072
Sierra Leone	-4.9082
Burundi	-4.8803
Niger	-4.5575

Table 12: Countries with lowest PC1 scores (least developed)

A.8.3 Highest PC2 Values

Country	PC2 Score
Marshall Islands	2.9551
West Bank & Gaza	2.9118
Greece	2.8411
Djibouti	2.8250
Aruba	2.7564

Table 13: Countries with highest PC2 scores

A.8.4 Lowest PC2 Values

Country	PC2 Score
Qatar	-6.4155
Kuwait	-3.6897
Brunei Darussalam	-3.1523
Oman	-2.8938
Bahrain	-2.8405

Table 14: Countries with lowest PC2 scores

B Original PCA Implementation

This appendix contains the original R script used for PCA analysis of World Development Indicators.

```
1 # PCA Analysis of World Development Indicators
2
3 library(tidyverse)
4 library(factoextra)
5 library(mice)
6 library(knitr)
7 library(ggrepel)
8 library(ggforce)
9 library(countrycode)
10
11 set.seed(123)
12
13 # Load data
14 wdi_data = DATA
15
16 # Clean column names
17 names(wdi_data) = gsub(" ", "_", names(wdi_data))
18 names(wdi_data) = gsub("\\[|\\]", "", names(wdi_data))
19 names(wdi_data) = gsub(",", "", names(wdi_data))
20 names(wdi_data) = gsub("\\$", "", names(wdi_data))
21
22 # Rename key columns
23 wdi_analysis = wdi_data %>%
24   rename(
25     country = Country_Name,
26     iso2c = Country_Code
27   ) %>%
28   select(-Time, -Time_Code)
29
30 # Get indicators
31 indicators = names(wdi_analysis)[!names(wdi_analysis) %in% c("country", "iso2c",
32   ")]
33
34 # Convert to numeric
35 wdi_analysis = wdi_analysis %>%
36   mutate(across(all_of(indicators), ~as.numeric(as.character(ifelse(. == "..",
37     NA, .)))))
38
39 # Filter countries with enough data
40 wdi_filtered = wdi_analysis %>%
41   mutate(missing_count = rowSums(is.na(select(., all_of(indicators))))) %>%
42   filter(missing_count <= length(indicators) * 0.3) %>%
43   select(-missing_count)
44
45 # Impute missing values
46 if(anyNA(wdi_filtered[, indicators])) {
47   imputed_data = mice(wdi_filtered[, indicators], m = 5, method = "pmm", seed
48     = 123, printFlag = FALSE)
49   wdi_complete = wdi_filtered
```

```

47 wdi_complete[, indicators] = complete(imputed_data)
48 } else {
49 wdi_complete = wdi_filtered
50 }
51
52 # Continent mapping
53 wdi_complete$continent_iso = countrycode(sourcevar = wdi_complete$iso2c,
54                                           origin = "iso2c",
55                                           destination = "continent",
56                                           warn = FALSE)
57
58 manual_continent_mapping = c(
59   # Regional groupings - don't assign to regular continents
60   "Africa Eastern and Southern" = NA,
61   "Africa Western and Central" = NA,
62   "Arab World" = NA,
63
64   # Specific countries requiring manual assignment
65   "American Samoa" = "Oceania",
66   "Andorra" = "Europe",
67   "Aruba" = "Americas",
68   "Austria" = "Europe",
69   "Azerbaijan" = "Asia",
70   "Bahamas, The" = "Americas",
71   "Bahrain" = "Asia",
72   "Bangladesh" = "Asia",
73   "Belarus" = "Europe",
74   "Belgium" = "Europe",
75   "Belize" = "Americas",
76   "Benin" = "Africa",
77
78   # Special cases
79   "Kosovo" = "Europe",
80   "Taiwan, China" = "Asia",
81   "Macao SAR, China" = "Asia",
82   "Hong Kong SAR, China" = "Asia",
83   "West Bank and Gaza" = "Asia",
84   "Puerto Rico" = "Americas",
85   "Curacao" = "Americas",
86   "Sint Maarten (Dutch part)" = "Americas",
87   "Virgin Islands (U.S.)" = "Americas",
88   "Guam" = "Oceania",
89   "Northern Mariana Islands" = "Oceania",
90   "French Polynesia" = "Oceania",
91   "New Caledonia" = "Oceania",
92   "Faroe Islands" = "Europe",
93   "Gibraltar" = "Europe",
94   "Isle of Man" = "Europe",
95   "Channel Islands" = "Europe",
96   "Yemen, Rep." = "Asia",
97   "Korea, Rep." = "Asia",
98   "Korea, Dem. People's Rep." = "Asia",
99   "Congo, Rep." = "Africa",

```

```

100   "Congo, Dem. Rep." = "Africa"
101 )
102
103 wdi_complete$continent = manual_continent_mapping[wdi_complete$country]
104
105 wdi_complete$continent = ifelse(is.na(wdi_complete$continent),
106                                wdi_complete$continent_iso,
107                                wdi_complete$continent)
108
109
110 for(i in which(is.na(wdi_complete$continent))) {
111   clean_name = str_replace(wdi_complete$country[i], ", The$", "")
112   clean_name = str_replace(clean_name, " SAR.*$", "")
113
114   continent_guess = countrycode(sourcevar = clean_name,
115                                 origin = "country.name",
116                                 destination = "continent",
117                                 warn = FALSE)
118
119   if(!is.na(continent_guess)) {
120     wdi_complete$continent[i] = continent_guess
121   }
122 }
123
124 # Check how many are still NA (should exclude regional groupings)
125 na_count = sum(is.na(wdi_complete$continent))
126 print(paste("Countries with NA continent:", na_count))
127 print("Countries without continent:")
128 print(wdi_complete$country[is.na(wdi_complete$continent)])
129
130 # For any remaining individual countries, make educated guesses
131 if(na_count > 0) {
132   remaining_na = wdi_complete$country[is.na(wdi_complete$continent)]
133
134   # Final fallback based on ISO codes
135   for(i in which(is.na(wdi_complete$continent))) {
136     code = wdi_complete$iso2c[i]
137     if(!is.na(code)) {
138       if(code %in% c("AFE", "AFW", "ARB")) {
139         # Regional groupings - leave as NA
140         next
141       } else if(code %in% c("AND", "AUT", "BEL", "BLR", "BGR", "BIH", "CHE", "
142         CZE", "DEU", "DNK", "ESP", "EST", "FIN", "FRA", "GBR", "GRC", "HRV",
143         "HUN", "IRL", "ISL", "ITA", "LIE", "LTU", "LUX", "LVA", "MCO", "MDA",
144         "MKD", "MLT", "MNE", "NLD", "NOR", "POL", "PRT", "ROU", "RUS", "SMR"
145         , "SRB", "SVK", "SVN", "SWE", "UKR", "VAT")) {
146       wdi_complete$continent[i] = "Europe"
147     } else if(code %in% c("AFG", "ARE", "ARM", "AZE", "BGD", "BHR", "BRN", "
148       BTN", "CHN", "GEO", "HKG", "IDN", "IND", "IRN", "IRQ", "ISR", "JPN",
149       "JOR", "KAZ", "KGZ", "KHM", "KOR", "KWT", "LAO", "LBN", "LKA", "MAC",
150       "MDV", "MMR", "MNG", "MYS", "NPL", "OMN", "PAK", "PHL", "PRK", "PSE"
151       , "QAT", "SAU", "SGP", "SYR", "THA", "TJK", "TKM", "TLS", "TUR", "TWN"
152       , "UZB", "VNM", "YEM")) {

```

```

144     wdi_complete$continent[i] = "Asia"
145 } else if(code %in% c("AGO", "BDI", "BEN", "BFA", "BWA", "CAF", "CIV", "
    CMR", "COD", "COG", "COM", "CPV", "DJI", "DZA", "EGY", "ERI", "ETH",
    "GAB", "GHA", "GIN", "GMB", "GNB", "GNQ", "KEN", "LBR", "LBY", "LSO",
    "MAR", "MDG", "MLI", "MOZ", "MRT", "MUS", "MWI", "NAM", "NER", "NGA"
    , "RWA", "SDN", "SEN", "SLE", "SOM", "SSD", "STP", "SWZ", "SYC", "TCD
    ", "TGO", "TUN", "TZA", "UGA", "ZAF", "ZMB", "ZWE")) {
146     wdi_complete$continent[i] = "Africa"
147 } else if(code %in% c("ABW", "AIA", "ARG", "ATG", "BHS", "BLZ", "BOL", "
    BRA", "BRB", "CAN", "CAY", "CHL", "COL", "CRI", "CUB", "CUW", "CYM",
    "DMA", "DOM", "ECU", "GLP", "GRD", "GTM", "GUY", "HND", "HTI", "JAM",
    "KNA", "LCA", "MAF", "MEX", "MSR", "MTQ", "NIC", "PAN", "PER", "PRI"
    , "PRY", "SLV", "SUR", "SXM", "TCA", "TTO", "URY", "USA", "VCT", "VEN
    ", "VGB", "VIR")) {
148     wdi_complete$continent[i] = "Americas"
149 } else if(code %in% c("ASM", "AUS", "COK", "FJI", "FSM", "GUM", "KIR", "
    MHL", "NCL", "NFK", "NIU", "NRU", "NZL", "PYF", "PYL", "SLB", "TKL",
    "TON", "TUV", "UMI", "VUT", "WLF", "WSM")) {
150     wdi_complete$continent[i] = "Oceania"
151 }
152 }
153 }
154 }
155
156 # Principal Component Analysis
157 wdi_scaled = scale(wdi_complete[, indicators])
158 rownames(wdi_scaled) = wdi_complete$country
159
160 pca_result = prcomp(wdi_scaled, scale. = FALSE)
161
162 # Flip PC2
163 pca_result$rotation[, 2] = -pca_result$rotation[, 2]
164 pca_result$x[, 2] = -pca_result$x[, 2]
165
166 # Extract variance information
167 variance_explained = summary(pca_result)$importance[2, ] * 100
168 cumulative_variance = summary(pca_result)$importance[3, ] * 100
169
170 # Short names for variables
171 var_names = names(wdi_complete[, indicators])
172 short_names = c(
173     "GDP per capita", "Life expectancy", "Tertiary education",
174     "Measles immunization", "Urban population", "CO2 emissions",
175     "Electricity access", "Internet usage", "Unemployment",
176     "Agriculture % GDP", "Industry % GDP", "Inflation"
177 )
178 var_mapping = setNames(short_names, var_names)
179
180 # Loadings and coordinates for variables
181 loadings_df <- as.data.frame(pca_result$rotation)
182 loadings_df$variables <- rownames(loadings_df)
183 loadings_df$short_names <- var_mapping[loadings_df$variables]
184

```

```

185 # Calculate contribution
186 var_cos2 <- get_pca_var(pca_result)$cos2
187 loadings_df$contrib <- rowSums(var_cos2[, 1:2]) * 100 / sum(var_cos2[, 1:2])
188
189 # Variables plot
190 var_plot <- ggplot(loadings_df, aes(x = PC1, y = PC2)) +
191   geom_circle(aes(x0 = 0, y0 = 0, r = 1), color = "gray65", fill = NA) +
192   geom_segment(aes(x = 0, y = 0, xend = PC1, yend = PC2, color = contrib),
193     arrow = arrow(length = unit(0.2, "cm")), size = 0.8) +
194   geom_text_repel(aes(label = short_names, color = contrib),
195     size = 3.5,
196     segment.alpha = 0.8,
197     max.overlaps = 12,
198     box.padding = 0.5,
199     min.segment.length = 0.1) +
200   scale_color_gradient2(low = "#00AFBB", mid = "#E7B800", high = "#FC4E07",
201     midpoint = median(loadings_df$contrib)) +
202   geom_hline(yintercept = 0, linetype = "dashed", color = "gray50") +
203   geom_vline(xintercept = 0, linetype = "dashed", color = "gray50") +
204   coord_fixed() +
205   theme_minimal() +
206   labs(title = "Variables Factor Map",
207     x = paste0("Dim1 (", round(variance_explained[1], 1), "%)"),
208     y = paste0("Dim2 (", round(variance_explained[2], 1), "%)"),
209     color = "contrib") +
210   theme(
211     plot.title = element_text(size = 14, face = "bold"),
212     plot.subtitle = element_text(size = 12),
213     legend.position = "right"
214   )
215
216 # Countries Plot with Continental Color Coding
217 country_pca = as.data.frame(pca_result$x)
218 country_pca$country = rownames(country_pca)
219 country_pca$continent = wdi_complete$continent[match(country_pca$country, wdi_
220   complete$country)]
221
222 # Remove regional groupings with NA continents
223 country_pca_filtered = country_pca %>%
224   filter(!is.na(continent))
225
226 continent_colors = c(
227   "Africa" = "#E41A1C",
228   "Asia" = "#377EB8",
229   "Europe" = "#4DAF4A",
230   "Oceania" = "#984EA3",
231   "Americas" = "#FF7F00"
232 )
233
234 # Select representative countries
235 countries_to_label = c(
236   country_pca_filtered %>% arrange(desc(PC1)) %>% head(5) %>% pull(country),
237   country_pca_filtered %>% arrange(PC1) %>% head(5) %>% pull(country),

```

```

237 country_pca_filtered %>% arrange(desc(PC2)) %>% head(3) %>% pull(country),
238 country_pca_filtered %>% arrange(PC2) %>% head(3) %>% pull(country)
239 )
240
241 # Country plot with continent color coding
242 country_plot = ggplot(country_pca_filtered, aes(x = PC1, y = PC2, color =
  continent)) +
243   geom_point(size = 2, alpha = 0.7) +
244   geom_text_repel(
245     data = country_pca_filtered %>% filter(country %in% countries_to_label),
246     aes(label = country),
247     size = 3,
248     box.padding = 0.35,
249     point.padding = 0.5,
250     segment.color = "grey50",
251     max.overlaps = 40
252   ) +
253   scale_color_manual(values = continent_colors, na.value = "gray70") +
254   theme_minimal() +
255   labs(
256     title = "Countries Projected onto Principal Components",
257     x = paste0("PC1 (", round(variance_explained[1], 1), "% of variance)"),
258     y = paste0("PC2 (", round(variance_explained[2], 1), "% of variance)"),
259     color = "Continent"
260   ) +
261   theme(
262     legend.position = "right",
263     plot.title = element_text(size = 14, face = "bold"),
264     plot.subtitle = element_text(size = 12)
265   )
266
267 # Scree Plot
268 scree_plot = fviz_eig(pca_result,
269   addlabels = TRUE,
270   barfill = "#4682B4",
271   barcolor = "#4682B4",
272   linecolor = "#363636",
273   ggtheme = theme_minimal()) +
274   labs(title = "Scree Plot: Explained Variance by Principal Components",
275     subtitle = paste0("First two components explain ",
276       round(cumulative_variance[2], 1), "% of total
277         variance")) +
278   theme(
279     plot.title = element_text(size = 14, face = "bold"),
280     plot.subtitle = element_text(size = 12),
281     axis.title = element_text(size = 12)
282   )
283
284 # Display plots
285 print(scree_plot)
286 print(var_plot)
287 print(country_plot)

```

```

288 # Summary
289
290 # Check continent distribution
291 cat("\nContinent distribution:\n")
292 print(table(country_pca_filtered$continent, useNA = "ifany"))
293
294 cat("\nTop 5 most developed countries (highest PC1):\n")
295 top_countries = country_pca_filtered %>% arrange(desc(PC1)) %>% head(5)
296 for(i in 1:5) {
297     cat(sprintf("%s (%s): %.2f\n", top_countries$country[i], top_countries$
298         continent[i], top_countries$PC1[i]))
299 }
300
301 cat("\nTop 5 least developed countries (lowest PC1):\n")
302 bottom_countries = country_pca_filtered %>% arrange(PC1) %>% head(5)
303 for(i in 1:5) {
304     cat(sprintf("%s (%s): %.2f\n", bottom_countries$country[i], bottom_countries
305         $continent[i], bottom_countries$PC1[i]))
306 }

```

Listing 1: Original PCA Script

C Manual PCA Implementation

This appendix contains the R script for the manual implementation of PCA, without using built-in functions.

```
1 # PCA Analysis of World Development Indicators
2
3 library(tidyverse)
4 library(mice)
5
6 set.seed(123)
7
8 wdi_data = DATA
9
10 # Clean column names
11 names(wdi_data) = gsub(" ", "_", names(wdi_data))
12 names(wdi_data) = gsub("\\[|\\]", "", names(wdi_data))
13 names(wdi_data) = gsub(",", "", names(wdi_data))
14 names(wdi_data) = gsub("\\$", "", names(wdi_data))
15
16 wdi_analysis = wdi_data %>%
17   rename(
18     country = Country_Name,
19     iso2c = Country_Code
20   ) %>%
21   select(-Time, -Time_Code)
22
23 indicators = names(wdi_analysis)[!names(wdi_analysis) %in% c("country", "iso2c",
24   ")]
25
26 wdi_analysis = wdi_analysis %>%
27   mutate(across(all_of(indicators), ~as.numeric(as.character(ifelse(. == "..",
28     NA, .)))))
29
30 # Only keep countries with at least 70% of indicators available
31 wdi_filtered = wdi_analysis %>%
32   mutate(missing_count = rowSums(is.na(select(., all_of(indicators))))) %>%
33   filter(missing_count <= length(indicators) * 0.3) %>%
34   select(-missing_count)
35
36 # Impute remaining missing values using MICE
37 if(anyNA(wdi_filtered[, indicators])) {
38   imputed_data = mice(wdi_filtered[, indicators], m = 5, method = "pmm", seed
39     = 123, printFlag = FALSE)
40   wdi_complete = wdi_filtered
41   wdi_complete[, indicators] = complete(imputed_data)
42 } else {
43   wdi_complete = wdi_filtered
44 }
45
46 # =====
47 # Principal Component Analysis
48 # =====
```

```

46
47 # Step 1: Extract numeric data matrix
48 print("STEP 1: Extracting and preparing data matrix")
49 X = as.matrix(wdi_complete[, indicators])
50 rownames(X) = wdi_complete$country
51 n = nrow(X)
52 p = ncol(X)
53 print(paste("Data matrix dimensions:", n, "countries", p, "indicators"))
54
55 # Step 2: Center and scale the data (standardization)
56 print("STEP 2: Standardizing data (centering and scaling)")
57 X_means = colMeans(X)
58 X_sds = apply(X, 2, sd)
59 X_scaled = scale(X, center = TRUE, scale = TRUE)
60 print("First few means of indicators:")
61 print(head(X_means))
62 print("First few standard deviations of indicators:")
63 print(head(X_sds))
64 print("Data has been centered (mean = 0) and scaled (sd = 1) for each variable
    ")
65
66 # Step 3: Compute the covariance matrix
67 print("STEP 3: Computing the correlation/covariance matrix")
68 cov_matrix = (t(X_scaled) %*% X_scaled) / (n - 1)
69 print("Dimensions of covariance matrix:")
70 print(dim(cov_matrix))
71 print("First 3x3 section of the covariance matrix:")
72 print(cov_matrix[1:3, 1:3])
73
74 # Step 4: Compute eigenvalues and eigenvectors of the covariance matrix
75 print("STEP 4: Computing eigenvalues and eigenvectors")
76 eigen_result = eigen(cov_matrix)
77 eigenvalues = eigen_result$values
78 eigenvectors = eigen_result$vectors
79
80 # Check for any numerical issues
81 if(any(eigenvalues < 0 & abs(eigenvalues) < 1e-10)) {
82     eigenvalues[eigenvalues < 0 & abs(eigenvalues) < 1e-10] = 0
83 }
84
85 print("First few eigenvalues:")
86 print(head(eigenvalues))
87
88 # Step 5: Calculate variance explained and cumulative variance
89 print("STEP 5: Calculating variance explained by each principal component")
90 total_variance = sum(eigenvalues)
91 variance_explained = eigenvalues / total_variance * 100
92 cumulative_variance = cumsum(variance_explained)
93
94 print("Proportion of variance explained by each PC:")
95 var_table = data.frame(
96     PC = 1:length(eigenvalues),
97     Eigenvalue = eigenvalues,

```

```

98     Proportion = variance_explained,
99     Cumulative = cumulative_variance
100 )
101 print(head(var_table, 10))
102
103 # Step 6: Construct loadings matrix (eigenvectors)
104 print("STEP 6: Creating the loadings matrix (eigenvectors)")
105 loadings = eigenvectors
106 colnames(loadings) = paste0("PC", 1:p)
107 rownames(loadings) = colnames(X)
108
109 # FLIP PC1 TO CORRECT INTERPRETATION (higher PC1 = more developed)
110 loadings[, 1] = -loadings[, 1]
111
112 print("First few loadings for the first 3 principal components:")
113 print(loadings[1:5, 1:3])
114
115 # Step 7: Project the data onto principal components
116 print("STEP 7: Projecting the data onto principal components")
117 scores = X_scaled %*% loadings
118 colnames(scores) = paste0("PC", 1:p)
119
120 print("First few rows of principal component scores:")
121 print(head(scores[, 1:3]))
122
123 # Step 8: Interpreting the principal components
124 print("STEP 8: Interpreting the first two principal components")
125
126 # Display loadings for PC1 sorted by absolute value
127 pc1_loadings = loadings[, 1]
128 pc1_sorted = sort(abs(pc1_loadings), decreasing = TRUE)
129 print("Top contributors to PC1 (by absolute loading):")
130 for(var in names(head(pc1_sorted, 5))) {
131     print(paste0(var, ": ", round(pc1_loadings[var], 4),
132               " (", ifelse(pc1_loadings[var] > 0, "positive", "negative"), "
133               correlation)"))
134     print(paste0("    Interpretation: ",
135               ifelse(pc1_loadings[var] > 0,
136                     "Higher values increase PC1 (indicates more development)
137                     ",
138                     "Lower values increase PC1 (indicates more development)"
139               )))
140 }
141
142 # Display loadings for PC2 sorted by absolute value
143 pc2_loadings = loadings[, 2]
144 pc2_sorted = sort(abs(pc2_loadings), decreasing = TRUE)
145 print("\nTop contributors to PC2 (by absolute loading):")
146 for(var in names(head(pc2_sorted, 5))) {
147     print(paste0(var, ": ", round(pc2_loadings[var], 4),
148               " (", ifelse(pc2_loadings[var] > 0, "positive", "negative"), "
149               correlation)"))
150 }

```

```

147
148 # Step 9: Identifying extreme countries
149 print("STEP 9: Identifying countries at extremes of first two principal
      components")
150
151 # Top countries on PC1 (MOST developed)
152 pc1_top = head(scores[order(scores[, 1], decreasing = TRUE), 1, drop = FALSE],
      5)
153 pc1_bottom = head(scores[order(scores[, 1]), 1, drop = FALSE], 5)
154
155 print("Countries with highest values on PC1 (MOST developed):")
156 for(i in 1:nrow(pc1_top)) {
157   print(paste0(rownames(pc1_top)[i], ": ", round(pc1_top[i, 1], 4)))
158 }
159
160 print("Countries with lowest values on PC1 (LEAST developed):")
161 for(i in 1:nrow(pc1_bottom)) {
162   print(paste0(rownames(pc1_bottom)[i], ": ", round(pc1_bottom[i, 1], 4)))
163 }
164
165 # Top countries on PC2
166 pc2_top = head(scores[order(scores[, 2], decreasing = TRUE), 2, drop = FALSE],
      5)
167 pc2_bottom = head(scores[order(scores[, 2]), 2, drop = FALSE], 5)
168
169 print("Countries with highest values on PC2:")
170 for(i in 1:nrow(pc2_top)) {
171   print(paste0(rownames(pc2_top)[i], ": ", round(pc2_top[i, 1], 4)))
172 }
173
174 print("Countries with lowest values on PC2:")
175 for(i in 1:nrow(pc2_bottom)) {
176   print(paste0(rownames(pc2_bottom)[i], ": ", round(pc2_bottom[i, 1], 4)))
177 }

```

Listing 2: Manual PCA Implementation